

Desarrollo de un Sistema Automatizado en Entorno Bash para el Control de Recursos de
Hardware

Juan L. Madueño

Facultad de Ingeniería de Sistemas e Informática, Universidad Nacional Mayor de San

Marcos

201230301 - Introducción a la Computación

Mg. Gelber C. Uscuchagua

23 de mayo de 2026

Índice

Introducción.....	3
Objetivos del script.....	4
Arquitectura lógica y principios de sistemas.....	4
Atributos clave del diseño del software:.....	4
Detalle técnico de funcionamiento.....	5
Configuración de variables iniciales.....	5
Gestión visual mediante paletas de colores ANSI.....	5
Análisis lógico del subsistema de almacenamiento en disco.....	7
Análisis lógico del subsistema de memoria volátil (RAM).....	8
Análisis lógico del subsistema de tiempo de actividad (uptime).....	9
Módulo de control de retroalimentación y cierre del sistema.....	10
Conclusión.....	11

Introducción

En el ámbito de la ingeniería de sistemas y la administración de servidores, la supervisión de la infraestructura es un pilar crítico para garantizar la alta disponibilidad y la continuidad operativa. La degradación no controlada de los recursos físicos (como el desbordamiento de memoria volátil o la saturación del almacenamiento persistente) produce fallos catastróficos en cascada.

Este informe documenta el diseño e implementación del monitor de recursos automatizado, una solución nativa basada en la automatización Shell (Bash). Esta herramienta actúa como un monitor de telemetría liviano y en tiempo real que opera directamente sobre la capa del kernel Linux a través de utilidades nativas del sistema. De este modo, recolecta métricas esenciales de rendimiento sin necesidad de instalar software de terceros o agentes pesados, cumpliendo así con los principios de eficiencia e interoperabilidad requeridos en la administración de servidores profesionales.

Objetivos del script

Monitoreo proactivo

Evaluar continuamente los indicadores clave de rendimiento (KPI) del hardware del servidor: almacenamiento persistente, memoria RAM y el tiempo de actividad acumulado (*uptime*).

Gestión defensiva de alertas

Clasificar de manera visual e inmediata el estado de los componentes del sistema mediante códigos de color ANSI (estables en verde, alertas críticas en rojo), permitiendo una rápida auditoría visual por parte del operador del sistema.

Persistencia de eventos (logging)

Proveer un registro histórico local e inmutable (`registro_sistema.log`) que almacene de manera precisa las marcas de tiempo e incidentes de desbordamiento de umbrales, fundamental para el análisis posterior a fallos (*post-mortem*) y auditorías de TI.

Arquitectura lógica y principios de sistemas

Desde la perspectiva de la Teoría General de Sistemas (TGS), el script se comporta como un mecanismo de retroalimentación negativa (homeostasis) para un entorno de servidores. El script lee las salidas informativas del sistema operativo, las procesa internamente comparándolas con límites preestablecidos, y genera una salida informativa y un registro en disco para que el administrador tome acciones correctivas antes de que el sistema colapse por entropía.

Atributos clave del diseño del software:

Modularidad estricta. Las rutinas de monitoreo están claramente delimitadas en tres subsistemas (Disco, RAM, Tiempo de Actividad), facilitando el mantenimiento futuro o la inserción de nuevos sensores métricos (como el uso de CPU o tráfico de red).

Uso de pipelines y parseo predictable. El código saca el máximo provecho al flujo secuencial de Linux (`'stdout'` y `'stdin'`) encadenando comandos nativos de diagnóstico a través de tuberías (`|`) hacia herramientas de procesamiento de texto estructurado como `'awk'` y `'sed'`.

Detalle técnico de funcionamiento

El script ejecuta un flujo lineal segmentado en cuatro módulos operativos:

Configuración de variables iniciales

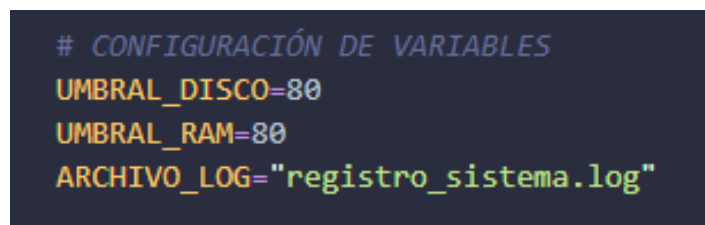
Como se puede observar en la Figura 1, el script comienza definiendo los parámetros clave para el control del servidor, que son `UMBRAL_DISCO=80`, `UMBRAL_RAM=80` y `ARCHIVO_LOG="registro_sistema.log"`. Estas líneas establecen de forma numérica el límite de tolerancia (80%) antes de que el monitor decida que el hardware está en una situación

crítica. Al mismo tiempo, se define el nombre del archivo de texto donde se guardará el historial de errores. En el lenguaje de programación Bash, este proceso se realiza de manera dinámica, lo que significa que el sistema reconoce automáticamente si el dato es un número o un texto sin necesidad de que el programador tenga que declararlo explícitamente al inicio.

Desde el punto de vista del diseño de software, colocar estos valores al principio del archivo es una excelente práctica que evita la codificación dura o ‘*hardcoding*’, que consiste en escribir números sueltos dentro de las funciones del script. Al centralizar los umbrales en este bloque, el código se vuelve mucho más fácil de mantener y escalar. Si un administrador de servidores decide en el futuro cambiar el límite de alerta al 90%, solo tendrá que modificar esa línea una sola vez en la cabecera, logrando que todas las condiciones lógicas de la parte inferior se adapten automáticamente al nuevo valor.

Figura 1

Bloque de inicialización y parametrización de variables globales en Bash



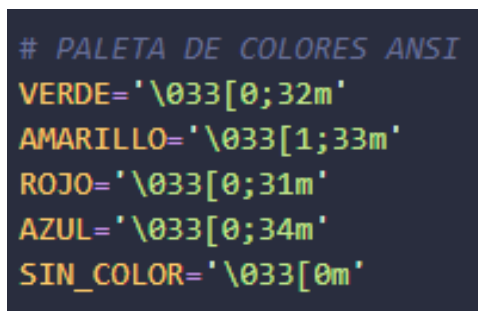
```
# CONFIGURACIÓN DE VARIABLES
UMBRAL_DISCO=80
UMBRAL_RAM=80
ARCHIVO_LOG="registro_sistema.log"
```

Gestión visual mediante paletas de colores ANSI

El diseño de la interfaz de usuario del script incorpora un sistema de alertas visuales basado en códigos de escape ANSI. Como se puede ver en la Figura 2, mediante variables como VERDE="\033[0;32m", AMARILLO="\033[1;33m", ROJO="\033[0;31m" y AZUL="\033[0;34m", el monitor asigna dinámicamente texturas cromáticas a los mensajes que se imprimen en la pantalla. Para cerrar correctamente cada bloque de color y evitar que el formato afecte al resto de la terminal, se utiliza la variable SIN_COLOR="\033[0m", que actúa como un restaurador del estado original del sistema. Esta estrategia facilita la interpretación

inmediata de la telemetría, permitiendo que el operador identifique problemas de un solo vistazo.

Figura 2



```
# PALETA DE COLORES ANSI
VERDE='\033[0;32m'
AMARILLO='\033[1;33m'
ROJO='\033[0;31m'
AZUL='\033[0;34m'
SIN_COLOR='\033[0m'
```

Declaración de variables cromáticas mediante códigos de escape ANSI

Interfaz visual y captura cronológica en tiempo real

El script inicia su ejecución desplegando una interfaz gráfica basada en texto para estructurar la presentación de los datos. Como se muestra en la Figura 3, el diseño utiliza el comando interno echo acompañado de la bandera -e, una opción esencial en sistemas tipo Unix que habilita la interpretación de caracteres especiales y secuencias de escape dentro de las cadenas impresas. En este bloque inicial, el monitor invoca la utilidad nativa date '+%Y-%m-%d %H:%M:%S', la cual realiza una llamada directa al reloj del sistema operativo para capturar e imprimir la marca temporal exacta del servidor en el momento justo del arranque. Esta técnica garantiza que cada ejecución cuente con un registro cronológico preciso, facilitando las tareas de auditoría y la correlación de eventos temporales por parte del administrador de la infraestructura.

Para lograr que los títulos y las alertas se destaquen visualmente en la pantalla, el script inyecta variables predefinidas como \${AZUL} al principio de los mensajes y las cierra con \${SIN_COLOR} al final. Este mecanismo funciona mediante el uso de códigos ANSI, que son estándares de comunicación que las terminales modernas interpretan para alterar propiedades estéticas del texto, como el color de la fuente o el fondo, sin interferir con los

datos lógicos del programa. Al concatenar estas variables con las salidas de texto, la terminal detiene temporalmente su renderizado por defecto para pintar los caracteres con el tono seleccionado, lo que permite clasificar jerárquicamente la información mostrada y asegurar que las alertas críticas resalten inmediatamente ante el ojo humano.

Figura 3

Módulo de despliegue de interfaz de usuario y captura del registro cronológico inicial

```
echo -e "${AZUL}===== ${SIN_COLOR}"
echo -e "${AZUL} SISTEMA AUTOMATIZADO DE MONITOREO DE SERVIDORES ${SIN_COLOR}"
echo -e "${AZUL}===== ${SIN_COLOR}"
echo "Fecha actual: $(date '+%Y-%m-%d %H:%M:%S')"
echo "-----"
```

Análisis lógico del subsistema de almacenamiento en disco

El núcleo operativo del monitor de recursos inicia con la evaluación del sistema de almacenamiento secundario. Como se presenta en la Figura 4, para capturar el estado del disco duro en tiempo real, el script ejecuta de manera encadenada una serie de herramientas avanzadas de filtrado mediante la instrucción `USO_DISCO=$(df -h / | awk 'NR==2 {print $5}' | tr -d '%')`. Este proceso arranca invocando el comando nativo `df -h /`, el cual interroga al sistema de archivos raíz del servidor para obtener métricas legibles de espacio disponible y ocupado. La salida de este comando se redirige inmediatamente mediante una tubería hacia la herramienta `awk 'NR==2 {print $5}'`, la cual tiene la tarea específica de ignorar las cabeceras del reporte, saltar a la segunda fila de información y aislar únicamente la quinta columna, donde se encuentra el porcentaje actual de ocupación del disco.

Debido a que las respuestas nativas del sistema operativo entregan el dato acompañado del símbolo de porcentaje (por ejemplo, 45%), el script utiliza el comando de traducción `tr -d '%'` para eliminar dicho carácter de forma definitiva. Este paso de limpieza es crítico en la programación de Bash, ya que el intérprete requiere trabajar con valores numéricos enteros puros en base diez para poder realizar evaluaciones de magnitud. Una vez

que la variable contiene solo los dígitos esenciales, el script abre una estructura de control condicional mediante el comando `if [ "$USO_DISCO" -ge "$UMBRAL_DISCO" ]`. El operador relacional `-ge` actúa comparando de forma booleana si el espacio ocupado es mayor o igual al límite del ochenta por ciento definido en la cabecera.

Si la condición se cumple, lo que significa que el servidor está en riesgo de saturación, el script desvía su flujo para ejecutar una doble acción preventiva. Por un lado, imprime una alerta visual en la pantalla del administrador utilizando la paleta de colores para destacar el estado crítico en tono rojo. Por otro lado, genera persistencia de datos mediante el operador de redirección y anexo `>>`, el cual escribe una nueva línea al final del archivo `registro_sistema.log` sin borrar el historial anterior. Esta bitácora automatizada combina la marca temporal exacta del servidor obtenida mediante el comando `date` junto con el porcentaje del fallo detectado. En caso de que el espacio se encuentre por debajo del límite, el flujo salta directamente al bloque del `else`, notificando de forma limpia en la terminal que el almacenamiento se mantiene en un estado estable y seguro.

Figura 4

Implementación de condicionales lógicas y tuberías para la evaluación del almacenamiento secundario

```
# 1. MONITOREO DEL DISCO DURO (Corrección Multiplataforma)
USO_DISCO=$(df -h / | awk 'NR==2 {print $5}' | tr -d '%')

echo -n "Uso de Disco: ${USO_DISCO}% -> "
if [ "$USO_DISCO" -ge "$UMBRAL_DISCO" ]; then
    echo -e "${ROJO}[CRÍTICO] Espacio en disco casi lleno.${SIN_COLOR}"
    echo "${date '+%Y-%m-%d %H:%M:%S'} - [CRÍTICO] DISCO al ${USO_DISCO}%" >> "$ARCHIVO_LOG"
else
    echo -e "${VERDE}[OK] Almacenamiento estable.${SIN_COLOR}"
fi
```

Análisis lógico del subsistema de memoria volátil (RAM)

El segundo módulo del script se encarga de supervisar de forma automatizada el estado de la memoria RAM del servidor. Como se detalla en la Figura 5, la captura del

consumo de este recurso se realiza en tiempo real mediante la instrucción `USO_RAM=$(free | awk '/Mem:/ {print int($3/$2 * 100)}')`. Este flujo operativo comienza invocando el comando nativo `free`, una utilidad del sistema operativo que interroga directamente al núcleo de Linux para obtener las métricas de memoria utilizada, libre y disponible en el equipo. Inmediatamente después, el pipeline redirige la salida de texto generada hacia la herramienta de procesamiento `awk`, la cual filtra la información buscando únicamente la línea que inicia con la etiqueta `Mem:`, ignorando por completo los datos relacionados con la memoria de intercambio o *swap*.

Una vez localizada la fila correcta, el script realiza una operación aritmética matemática directamente sobre el flujo de datos. Divide el valor de la tercera columna, que representa la memoria RAM actualmente utilizada por los procesos del servidor, entre la segunda columna, que equivale a la memoria total física disponibilizada en el hardware; posteriormente, multiplica el resultado por cien para calcular el porcentaje de ocupación. En este punto, se aplica la función interna `int()` para truncar los residuos decimales y asegurar un número entero limpio. Este paso es indispensable debido a que Bash no cuenta con soporte nativo para evaluar números con punto flotante en sus condicionales básicas, por lo que forzar un entero garantiza la estabilidad de la lógica del monitor.

Con el porcentaje ya procesado y guardado, el monitor evalúa el estado del recurso mediante la estructura condicional `if [ "$USO_RAM" -ge "$UMBRAL_RAM" ]`. Si el valor calculado es mayor o igual al ochenta por ciento establecido en la configuración inicial, el flujo del programa activa las alertas del sistema. Esta respuesta imprime un mensaje en la terminal pintado en color rojo para capturar la atención del administrador y, al mismo tiempo, añade un registro histórico detallado en el archivo `registro_sistema.log` utilizando el operador `>>`. Si el consumo se encuentra dentro de los rangos normales, el script salta hacia la sección

del else, mostrando en la pantalla un indicador de color verde que confirma que el almacenamiento volátil opera de manera estable y segura.

Figura 5

Módulo automatizado de evaluación aritmética y condicional de la memoria volátil RAM

```
# 2. MONITOREO DE LA MEMORIA RAM (Corrección Multiplataforma)
USO_RAM=$(free | awk '/Mem:/ {print int($3/$2 * 100)}')

echo -n "Uso de Memoria RAM: ${USO_RAM}% -> "
if [ "$USO_RAM" -ge "$UMBRAL_RAM" ]; then
    echo -e "${ROJO}[CRÍTICO] Alta demanda de memoria.${SIN_COLOR}"
    echo "$(date '+%Y-%m-%d %H:%M:%S') - [CRÍTICO] RAM al ${USO_RAM}%" >> "$ARCHIVO_LOG"
else
    echo -e "${VERDE}[OK] Memoria estable.${SIN_COLOR}"
fi
```

Análisis lógico del subsistema de tiempo de actividad (uptime)

El tercer módulo de diagnóstico del script está diseñado para extraer y formatear el tiempo total que el servidor ha permanecido encendido de forma ininterrumpida. Como se aprecia en la Figura 6, la captura y limpieza de esta métrica se realiza mediante la instrucción compleja `TIEMPO_ACTIVO=$(uptime | awk -F, '{print $1}' | sed 's/.*up //')`. El proceso inicia ejecutando el comando nativo `uptime`, una utilidad de Linux que interroga directamente al kernel para obtener un resumen rápido sobre la hora actual, el tiempo de actividad del sistema, el número de usuarios conectados y los promedios de carga de trabajo del procesador.

Debido a que la salida estándar de este comando entrega toda la información junta en una sola línea separada por comas, el script implementa un primer filtro utilizando la herramienta `awk -F, '{print $1}'`. El parámetro `-F`, es un modificador clave que altera el comportamiento por defecto de `awk`, obligándolo a usar la coma como un divisor de bloques en lugar de los espacios en blanco tradicionales. Al ordenar la impresión del primer fragmento mediante `{print $1}`, el programa aísla con éxito el bloque inicial del texto y

descarta de manera inmediata los contadores de usuarios y cargas de procesamiento que no corresponden a este módulo.

Una vez obtenido este primer segmento, el flujo se conecta mediante una última tubería hacia el editor de flujo sed 's/*.up //' con el objetivo de pulir el formato de salida. Esta herramienta aplica una expresión regular de sustitución donde el patrón *.up localiza dinámicamente cualquier cadena de caracteres que preceda y contenga la palabra "up " junto con su espacio posterior, reemplazándola por un vacío absoluto. Esta técnica de depuración elimina los residuos del texto inicial del sistema y conserva únicamente la medición de tiempo limpia. Finalmente, el valor depurado se almacena en la variable TIEMPO_ACTIVO y se imprime en la terminal utilizando la variable \${AMARILLO}, lo que asegura que el dato de disponibilidad resalte visualmente para el operador sin cargar con textos redundantes.

Figura 6

Módulo de extracción y depuración del tiempo de actividad ininterrumpida del servidor

```
# 3. TIEMPO DE ACTIVIDAD
TIEMPO_ACTIVO=$(uptime | awk -F, '{print $1}' | sed 's/*.up //')
echo -e "Tiempo de actividad del sistema: ${AMARILLO}${TIEMPO_ACTIVO}${SIN_COLOR}"
```

Módulo de control de retroalimentación y cierre del sistema

El segmento final del programa está diseñado para ofrecer una respuesta estructurada al administrador sobre el resultado global de la inspección de hardware. Como se puede observar en la Figura 7, este bloque de control utiliza la condicional if [-f "\$ARCHIVO_LOG"] para validar la existencia física del archivo de bitácoras en el sistema de almacenamiento. El operador unario -f cumple la función específica de verificar si la ruta guardada en la variable corresponde a un archivo regular que ya contiene datos. Esta validación es clave porque el archivo registro_sistema.log solo se genera o se modifica si

alguno de los subsistemas anteriores (disco o RAM) detectó que se superó el umbral crítico del ochenta por ciento durante el monitoreo.

Si la evaluación resulta verdadera, lo que confirma que se registraron anomalías en los recursos, el script desvía su ejecución para notificarle de forma explícita al operador la ubicación exacta del reporte mediante un mensaje formateado con la etiqueta visual `${AZUL}[LOG]`. Por el contrario, si el servidor operó de manera óptima durante toda la rutina, la condicional salta directamente hacia el bloque del `else`, imprimiendo una cadena de texto que confirma que no se registraron alertas críticas en esa ejecución. Finalmente, el programa imprime una última línea de cierre utilizando la paleta cromática, delimitando de manera limpia el término del proceso automatizado y devolviendo el control total a la terminal de Linux.

Figura 7

Módulo de verificación de persistencia de datos y cierre del sistema de monitoreo

```
# 4. CONTROL DE RETROALIMENTACIÓN
echo "-----"
if [ -f "$ARCHIVO_LOG" ]; then
    echo -e "${AZUL}[LOG]${SIN_COLOR} Eventos críticos registrados en: ${ARCHIVO_LOG}"
else
    echo "No se registraron alertas críticas en esta ejecución."
fi
echo -e "${AZUL}===== ${SIN_COLOR}"
```

Conclusión

El desarrollo e implementación de este script de automatización demuestra cómo el uso eficiente de las herramientas nativas de Linux permite construir soluciones de monitoreo robustas, livianas y de alta fidelidad sin la necesidad de instalar software de terceros o sobrecargar los recursos del servidor. Al conectar utilidades esenciales del kernel como `df`, `free` y `uptime` a través de tuberías (*pipelines*), y procesar los flujos de datos mediante `awk` y `sed`, se logra un diagnóstico preciso de la infraestructura en tiempo real. Este enfoque purista

no solo optimiza el rendimiento del sistema operativo al ejecutar hilos en paralelo por memoria, sino que también garantiza que el administrador cuente con telemetría limpia y legible de manera inmediata.

Asimismo, la arquitectura del script refleja la importancia de las buenas prácticas en el diseño de software para la administración de sistemas. La centralización y parametrización de las variables al inicio del archivo previene errores comunes de codificación dura o *hardcoding*, facilitando que la herramienta sea completamente escalable y adaptable a las políticas de cualquier centro de datos con solo modificar un par de valores. Finalmente, la integración de alertas visuales mediante códigos ANSI y la persistencia de datos a través de una bitácora automatizada de errores (`registro_sistema.log`) configuran un entorno de monitoreo defensivo eficaz. Esta combinación asegura que el operador identifique problemas críticos de un vistazo y mantenga un registro histórico confiable, clave para la auditoría, la prevención de caídas y la alta disponibilidad de los servicios en producción.